

Digital Voltmeter

LOGAN RASMUSSEN, SHAYMON MADRID, TANNER RAY, GAVIN
MIKESELL

Table of Contents

Introduction and Project Overview	2
Specifications	2
Required Equipment and Software.....	3
Peer Reviews	3
1 st Peer Review	3
2 nd Peer Review.....	4
Procedure	4
Project Code and Results	7
Clock Gen Code (Shaymon)	7
ADC Code (Shaymon)	8
ADC to mv (Tanner)	10
DMM Controller Code (Tanner)	11
Seven Segment Display (Gavin)	13
Top Module (Logan).....	15
Testbench (Tanner)	17
Testbench Simulation (Tanner)	19
Working Design.....	19
Conclusion.....	20

Table of Figures

Figure 1: Custom ADC/DAC board	3
Figure 2: Diagram of Voltmeter	6
Figure 3: Elaborated Design for Project	7
Figure 4: Simulation of Voltmeter	19
Figure 5: Voltmeter Reading 1.2V.....	19
Figure 6: Voltmeter Reading 2.4V	20

Introduction and Project Overview

Our project is to design and implement an FPGA-based Digital Multimeter using the Basys3 development board and a 12-bit ADC. The multimeter will continuously measure an analog input voltage, convert the sampled data into a digital value, perform the necessary arithmetic to calculate the voltage, and display the result on the Basys3's seven-segment display. A finite state machine (FSM) will control operating modes such as live voltage measurement, peak hold, and minimum and maximum voltage recording, while memory will store measurement data in real time. This project is important because digital multimeters are essential tools used throughout electrical engineering, manufacturing, research, and industrial automation for testing and troubleshooting electronic systems. The design will follow applicable industry specifications, including the timing requirements of the ADC interface, reliable operation at the Basys3's 100MHz system clock, and the measurement limits and resolution provided by the 12-bit ADC, resulting in accurate and real-time voltage measurements.

The following objectives of the lab are to be met:

1. Define a complex digital or mixed-signal system
2. Collaborate with others on your design
3. Implement a complex digital or mixed-signal system
4. Write a technical report

Specifications

- Continuously measure an analog input voltage
- Use the following technologies:
 - Finite State Machine (at least 3 states)
 - Real – Time Memory access at 100MHz
 - Higher level Math Functions (Multiply or Divide)
 - 7-segment display
 - External Sensors (light, sound, off-board switches)
 - Serial Interface to Other Board
 - Analog to Digital Converters (ADCs) or Digital to Analog Converters (DACs)
 - Voltage conversion logic
 - Peak / Min / Max measurement storage
 - Fixed-point arithmetic
 - Measurement mode control

Required Equipment and Software

- Vivado (Using system Verilog)
- BASYS 3 FPGA
- AD74760 ADC
- A DC Supply

Peer Reviews

Part of this project involved discussing the project and demonstrating it to peers and the instructor. The first peer review was done early in the project timeline and was with fellow students in the course but not in the group. The second peer review was done with our Instructor, Brother Randall Jack, where he gave some feedback on how the project could be expanded.

1st Peer Review

Those we had this peer review with are Jordan Nelson, Harrison Hodgson, and an observing instructor Brother Allred.

Some of the questions they proposed are what kind of ADC and hardware will we be using, and how we will be testing the system. We informed them that we will be using the ADC/DAC board (shown in Figure 1) created for this class rather than the built-in ADC on the FPGA due to how noisy the built-in one is.

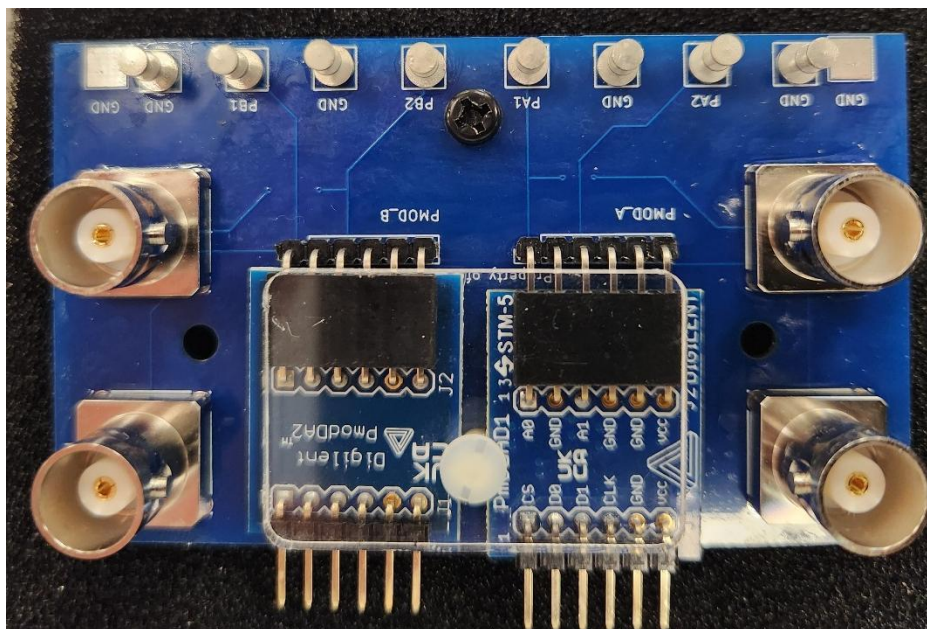


Figure 1: Custom ADC/DAC board

2nd Peer Review

Brother Randall Jack informed us that he liked our project. We discussed how we could try to add a AC RMS voltage reading as well so that it would be more applicable to more than just reading DC voltage. Other ideas that we had but were not able to implement include an option to measure AC voltages.

Procedure

1. Created a plan and timeline for the project.

The team met to determine the overall goals and requirements of the project. We decided to design a digital multimeter capable of measuring an analog voltage using an external 12-bit ADC and displaying the measured voltage on the Basys3 seven-segment display. After defining the project scope, we created a development timeline with milestones for completing modules, integrating the system, testing the design, and preparing the final report and demonstration.

2. Divided assignments between group members.

Improving our efficiency we each were assigned a module of the project to design. Individual responsibilities included developing the clock generation module, implementing the ADC interface, designing the voltmeter controller and memory functions, creating the seven-segment display driver, integrating the top-level module, and writing a simulation testbench. Throughout the project, team members met regularly to discuss progress, resolve design issues, and ensure all modules were compatible before integration.

3. Created a clock gen module.

A clock generation module was designed to derive the lower-frequency clocks required by the ADC from the Basys3's 100MHz system clock. This module generated the serial communication clock and sampling clock needed to operate the ADC according to its timing specifications.

4. Created an ADC.

The ADC module was developed to communicate with the external 12-bit ADC using a serial interface. The module shifted in the serial data from the ADC, assembled the 12-bit conversion result, and made the digital voltage value available to the remainder of the system.

5. Created Memory and voltmeter controller.

The controller module processed the ADC output by converting the digital signal into a voltage measurement. A finite state machine (FSM) controlled the operating modes of the multimeter, including live voltage measurement, peak hold, minimum, and maximum value display. Memory registers continuously stored the highest and lowest measured values while updating the displayed measurement in real time.

6. Created a seven-segment display.

The seven-segment display controller was implemented to display the measured voltage on the Basys3 board. The display driver converted binary voltage values into decimal digits and refreshed the four seven-segment displays fast enough to appear continuously illuminated.

7. Created top module.

After each module was completed individually, a top-level module was created to integrate all hardware components into one design. This module instantiated the clock generator, ADC interface, voltmeter controller, memory, and seven-segment display driver while routing the required signals between each module.

8. Created a testbench.

Simulation testbench was written for the integrated system. The testbench supplied simulated ADC data, verified state transitions within the finite state machine, confirmed proper voltage calculations, and ensured that the correct values were displayed under different operating conditions. Simulation allowed many design errors to be identified and corrected before hardware testing.

9. Tested with real circuit.

After successful simulation, the design was programmed onto the Basys3 FPGA and connected to the external ADC hardware. Known input voltages were applied and

compared against the displayed measurements to verify proper operation. Additional testing confirmed correct mode switching, accurate voltage conversion, memory operation for peak/minimum/maximum values, and reliable performance of the complete digital multimeter system.

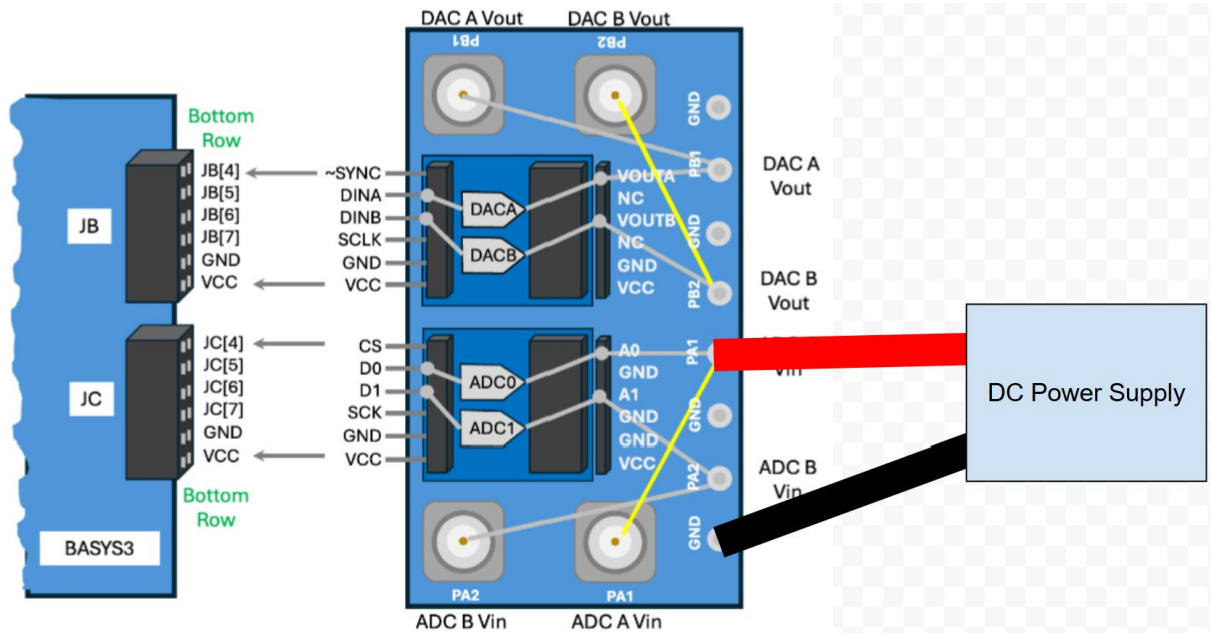


Figure 2: Diagram of Voltmeter

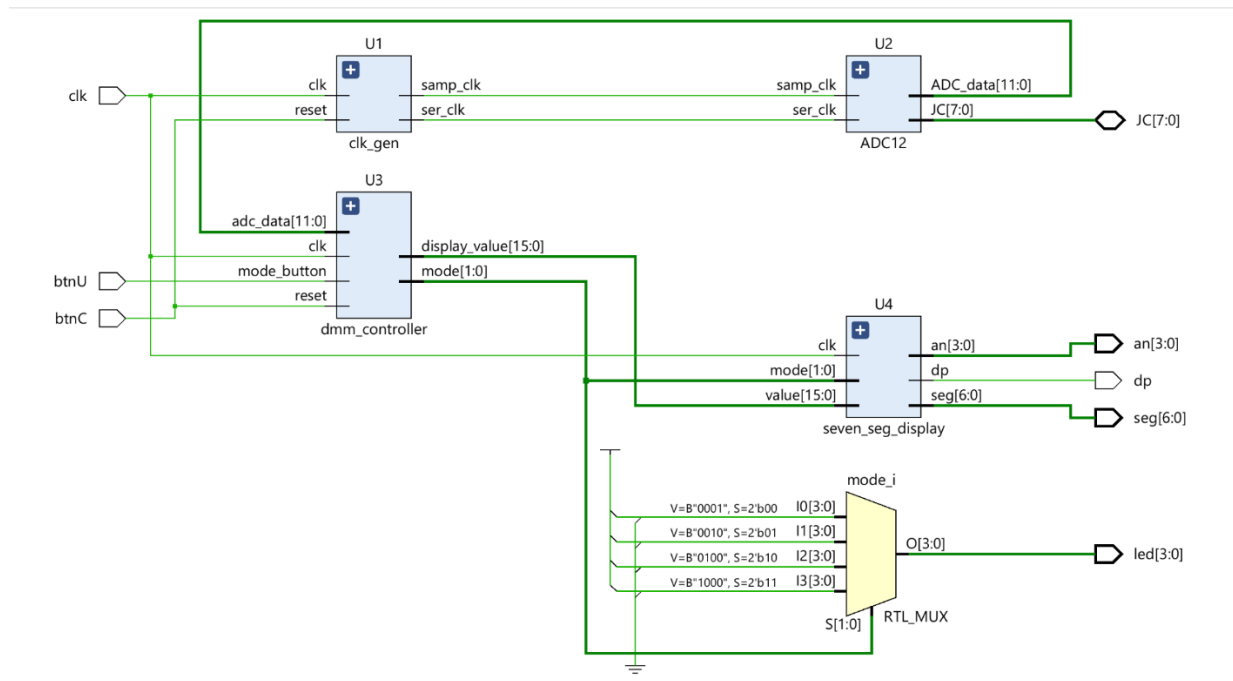


Figure 3: Elaborated Design for Project

Project Code and Results

Due to how this project was broken down amongst the group members, the code and results of simulation will be shown below. Each module is separated into its own section with the person who was responsible for creating/validating the module(s).

Clock Gen Code (Shaymon)

This module is used to generate the various clock signals needed to synchronize various parts of the project. It makes use of a divider for the 1 MHz clock and a PLL through use of Vivados IP source library.

```
`timescale 1ns / 1ps
module clk_gen (
    input wire clk,           // 100 MHz Basys3 system clock
    input wire reset,         // Active-high reset
    output wire ser_clk,      // 20 MHz serial clock for ADC
    output wire samp_clk     // 1 MHz sample clock for ADC
);
    #1000000000;
    ser_clk <-> 0;
    samp_clk <-> 0;
endmodule
```



```

// PLL-generated clock signals
wire clk_20MHz;
wire clk_10MHz;
wire clk_8MHz;

// Clocking Wizard IP instance
clk_wiz_0 pll_inst (
    .clk_in1(clk),
    .clk_out20MHz(clk_20MHz),
    .clk_out10MHz(clk_10MHz),
    .clk_out8MHz(clk_8MHz)
);

// Divide 8 MHz clock down to 1 MHz
reg [2:0] counter_8to1 = 3'b0;

always @(posedge clk_8MHz or posedge reset) begin
    if (reset)
        counter_8to1 <= 3'd0;
    else
        counter_8to1 <= counter_8to1 + 1'b1;
end

// Serial clock is directly generated by PLL
assign ser_clk = clk_20MHz;

// MSB of 3-bit counter creates 1 MHz square wave from 8
MHz clock
assign samp_clk = counter_8to1[2];
endmodule

```

ADC Code (Shaymon)

This module handles the interaction between the rest of the project and the ADC that we used in this lab. It takes input information and reference clocks to then read data from the ADC and output for use later in the project. This and the clk_gen module successfully passed individual testbenches prior to being sent to Tanner and Logan for final integration into the whole project.

```

`timescale 1ns / 1ps
module ADC12(
    input ser_clk,           // 20 MHz serial clock
    input samp_clk,          // 1 MHz sample clock
    inout [7:0] JC,          // ADC PMOD connector

```

```

    output reg [11:0] ADC_data // Captured 12-bit ADC value
);

    reg [4:0] ser_clk_cnt;
    wire [4:0] bit_number;
    reg CS;
    reg [19:0] data_Ch_A;
    reg samp_clk_tm1;

    // PMOD pin connections for the ADC board
    assign JC[7] = ser_clk; // ADC serial clock SCK
    assign JC[4] = CS; // ADC active-low chip select
CS

    // ADC data pins are inputs, so the FPGA should not drive
    them
    assign JC[6] = 1'bz; // ADC B data input, unused
    here
    assign JC[5] = 1'bz; // ADC A data input

    // Unused lower PMOD pins are left floating
    assign JC[3:0] = 4'bzzzz;

    assign bit_number = ser_clk_cnt;

    initial begin
        ser_clk_cnt = 5'd19;
        CS = 1'b1;
        data_Ch_A = 20'd0;
        ADC_data = 12'd0;
        samp_clk_tm1 = 1'b0;
    end

    // Read serial ADC data on the falling edge of the serial
    clock
    always @(negedge ser_clk) begin
        data_Ch_A[bit_number] <= JC[5];
    end

    // Save the completed 12-bit ADC value once per sample
    clock
    always @(posedge samp_clk) begin
        ADC_data <= data_Ch_A[11:0];
    end

    // Counter points to the serial data bit location
    always @(posedge ser_clk) begin

```

```

        samp_clk_tm1 <= samp_clk;

        if (samp_clk && !samp_clk_tm1)
            ser_clk_cnt <= 5'd18;
        else if (ser_clk_cnt > 0)
            ser_clk_cnt <= ser_clk_cnt - 1;
        else
            ser_clk_cnt <= 5'd19;
    end

    // Generate ADC chip select from the serial clock counter
    always @(*) begin
        case (ser_clk_cnt)
            5'd0:  CS = 1'b0;
            5'd1:  CS = 1'b0;
            5'd2:  CS = 1'b0;
            5'd3:  CS = 1'b0;
            5'd4:  CS = 1'b0;
            5'd5:  CS = 1'b0;
            5'd6:  CS = 1'b0;
            5'd7:  CS = 1'b0;
            5'd8:  CS = 1'b0;
            5'd9:  CS = 1'b0;
            5'd10: CS = 1'b0;
            5'd11: CS = 1'b0;
            5'd12: CS = 1'b0;
            5'd13: CS = 1'b0;
            5'd14: CS = 1'b0;
            5'd15: CS = 1'b0;
            5'd16: CS = 1'b1;
            5'd17: CS = 1'b1;
            5'd18: CS = 1'b1;
            5'd19: CS = 1'b1;
            default: CS = 1'b1;
        endcase
    end
endmodule

```

ADC to mv (Tanner)

This is a fairly simple module, its purpose is to take the data processed by the ADC12 module and convert the new digital data into relatable data that can be used for the seven segment displays.

```

`timescale 1ns / 1ps
module adc_to_mv(

```

```

    input [11:0] adc_count,
    output [15:0] voltage_mv
);

    // For 3.3 V ADC reference:
    // voltage_mv = adc_count * 3300 / 4095

    assign voltage_mv = (adc_count * 3300) / 4095;
endmodule

```

DMM Controller Code (Tanner)

This module is for controlling how the device behaves. It uses indicators via LEDs and buttons to determine the operating modes of the device. This module also handles some button debouncing.

```

`timescale 1ns / 1ps
module dmm_controller(
    input clk,                                // 100 MHz Basys3 system
    clock
    input reset,                              // Active-high reset
    input mode_button,                        // Raw pushbutton input used
    to change modes
    input [11:0] adc_data,                    // 12-bit ADC reading
    output reg [15:0] display_value, // Selected value sent to
    display in mV
    output reg [1:0] mode                    // Current display mode
);

    // Debounce and edge-detection signals for the mode button
    reg [20:0] debounce_count;
    reg btn_sync_0;
    reg btn_sync_1;
    reg btn_stable;
    reg btn_prev;

    // Stored ADC readings for measurement history
    reg [11:0] min_value;
    reg [11:0] max_value;

    // Converted millivolt values
    wire [15:0] live_voltage;
    wire [15:0] min_voltage;
    wire [15:0] max_voltage;

    // Convert ADC values to millivolts

```

```

adc_to_mv V0 (
    .adc_count(adc_data),
    .voltage_mv(live_voltage)
);

adc_to_mv V1 (
    .adc_count(min_value),
    .voltage_mv(min_voltage)
);

adc_to_mv V2 (
    .adc_count(max_value),
    .voltage_mv(max_voltage)
);

// One-clock pulse when the debounced button is pressed
wire btn_pressed = btn_stable && !btn_prev;

// Synchronize and debounce the pushbutton
always @(posedge clk) begin
    if (reset) begin
        debounce_count <= 0;
        btn_sync_0 <= 0;
        btn_sync_1 <= 0;
        btn_stable <= 0;
        btn_prev <= 0;
    end else begin
        btn_sync_0 <= mode_button;
        btn_sync_1 <= btn_sync_0;

        if (btn_sync_1 != btn_stable) begin
            debounce_count <= debounce_count + 1;

            if (debounce_count == 21'd2_000_000) begin
                btn_stable <= btn_sync_1;
                debounce_count <= 0;
            end
        end else begin
            debounce_count <= 0;
        end

        btn_prev <= btn_stable;
    end
end

// Mode control and measurement storage
always @(posedge clk) begin

```

```

        if (reset) begin
            mode <= 2'b00;
            min_value <= 12'hFFF;
            max_value <= 12'd0;
        end else begin
            if (btn_pressed)
                mode <= mode + 1'b1;

            if (adc_data < min_value)
                min_value <= adc_data;

            if (adc_data > max_value)
                max_value <= adc_data;
        end
    end

    // Select which converted voltage value goes to the display
    always @(*) begin
        case (mode)
            2'b00: display_value = live_voltage;
            2'b01: display_value = min_voltage;
            2'b10: display_value = max_voltage;
            2'b11: display_value = live_voltage;
            default: display_value = live_voltage;
        endcase
    end
endmodule

```

Seven Segment Display (Gavin)

For us to have usable information about what the device is connected to, we need a way to view it. This module accomplishes this need by displaying our current voltage reading to the seven segment displays.

```

`timescale 1ns / 1ps
module seven_seg_display(
    input clk,                // 100 MHz Basys3 system clock
    input [15:0] value,       // Voltage value in millivolts
    input [1:0] mode,         // Current DMM mode, available if
display logic needs it

    output reg [6:0] seg,     // Seven-segment cathode outputs
    output reg [3:0] an,      // Seven-segment anode enables
    output reg dp             // Decimal point output
);

```

```

// Refresh counter used for display multiplexing
reg [19:0] refresh_count;

always @(posedge clk) begin
    refresh_count <= refresh_count + 1;
end

// Select which digit is currently active
wire [1:0] digit_select;
assign digit_select = refresh_count[19:18];

reg [3:0] digit;

// Break millivolt value into decimal digits
wire [3:0] thousands;
wire [3:0] hundreds;
wire [3:0] tens;
wire [3:0] ones;

assign thousands = value / 1000;
assign hundreds  = (value % 1000) / 100;
assign tens      = (value % 100) / 10;
assign ones      = value % 10;

// Choose active digit and decimal point location
always @(*) begin
    case (digit_select)
        2'b00: begin
            an = 4'b1110;
            digit = ones;
            dp = 1'b1;
        end

        2'b01: begin
            an = 4'b1101;
            digit = tens;
            dp = 1'b1;
        end

        2'b10: begin
            an = 4'b1011;
            digit = hundreds;
            dp = 1'b1;
        end

        2'b11: begin
            an = 4'b0111;

```

```

        digit = thousands;
        dp = 1'b0;          // Active-low decimal point for
X.XXX format
    end

    default: begin
        an = 4'b1111;
        digit = 4'd0;
        dp = 1'b1;
    end
endcase
end

// Convert selected decimal digit to seven-segment pattern
always @(*) begin
    case (digit)
        4'd0: seg = 7'b1000000;
        4'd1: seg = 7'b1111001;
        4'd2: seg = 7'b0100100;
        4'd3: seg = 7'b0110000;
        4'd4: seg = 7'b0011001;
        4'd5: seg = 7'b0010010;
        4'd6: seg = 7'b0000010;
        4'd7: seg = 7'b1111000;
        4'd8: seg = 7'b0000000;
        4'd9: seg = 7'b0010000;
        default: seg = 7'b1111111;
    endcase
end
endmodule

```

Top Module (Logan)

With all of our submodules completed, Logan was able to tie everything together into a top module so they actually work with each other and perform the functionality that we desire.

```

`timescale 1ns / 1ps
module digital_multimeter_top(
    input clk,                // 100 MHz Basys3 clock
    input btnC,               // Reset button
    input btnU,               // Mode select button
    inout [7:0] JC,           // ADC PMOD connector
    output [6:0] seg,          // Seven-segment cathodes
    output [3:0] an,           // Seven-segment anodes
    output dp,                // Seven-segment decimal point
    output [3:0] led           // LEDs show current DMM mode

```



```

);

wire ser_clk;
wire samp_clk;
wire [11:0] adc_data;
wire [15:0] display_value;
wire [1:0] mode;

clk_gen U1 (
    .clk(clk),
    .reset(btnC),
    .ser_clk(ser_clk),
    .samp_clk(samp_clk)
);

ADC12 U2 (
    .ser_clk(ser_clk),
    .samp_clk(samp_clk),
    .JC(JC),
    .ADC_data(adc_data)
);

dmm_controller U3 (
    .clk(clk),
    .reset(btnC),
    .mode_button(btnU),
    .adc_data(adc_data),
    .display_value(display_value),
    .mode(mode)
);

seven_seg_display U4 (
    .clk(clk),
    .value(display_value),
    .mode(mode),
    .seg(seg),
    .an(an),
    .dp(dp)
);

assign led[0] = (mode == 2'b00); // Live voltage
assign led[1] = (mode == 2'b01); // Minimum voltage
assign led[2] = (mode == 2'b10); // Maximum voltage
assign led[3] = (mode == 2'b11); // Live voltage / unused
mode

endmodule

```

Testbench (Tanner)

We used a testbench to verify what our project was doing, and found it to be working as we expected. We then took this to the physical board to test using a DC power supply.

```
`timescale 1ns / 1ps
module digital_multimeter_top_tb;
    reg clk;
    reg btnC;
    reg btnU;
    wire [7:0] JC;
    wire [6:0] seg;
    wire [3:0] an;
    wire dp;
    wire [3:0] led;

    digital_multimeter_top DUT (
        .clk(clk),
        .btnC(btnC),
        .btnU(btnU),
        .JC(JC),
        .seg(seg),
        .an(an),
        .dp(dp),
        .led(led)
    );

    // 100 MHz Basys3 clock
    always #5 clk = ~clk;

    task set_adc_value;
        input [11:0] value;
        begin
            force DUT.adc_data = value;
            #5_000_000;
        end
    endtask

    task press_mode_button;
        begin
            btnU = 1'b1;
            #25_000_000;
            btnU = 1'b0;
            #25_000_000;
        end
    end
```

```

endtask

initial begin
    clk = 0;
    btnC = 1;
    btnU = 0;

    #1_000;
    btnC = 0;

    // Apply ADC test values directly to verify DMM logic
    set_adc_value(12'd1241);    // About 1.000 V
    set_adc_value(12'd2482);    // About 2.000 V
    set_adc_value(12'd620);     // About 0.500 V

    // View live voltage mode
    #10_000_000;

    // Switch to minimum voltage mode
    press_mode_button;
    #10_000_000;

    // Switch to maximum voltage mode
    press_mode_button;
    #10_000_000;

    // Switch to unused/live mode
    press_mode_button;
    #10_000_000;

    $stop;
end
endmodule

// Simulation-only clock wizard replacement.
// This is only used by the testbench and does not affect
// hardware.
module clk_wiz_0(
    input clk_in1,
    output reg clk_out20MHz,
    output reg clk_out10MHz,
    output reg clk_out8MHz
);

    initial begin
        clk_out20MHz = 0;
        clk_out10MHz = 0;
    end
endmodule

```

```

        clk_out8MHz = 0;
    end

    always #25    clk_out20MHz = ~clk_out20MHz;    // 20 MHz
    always #50    clk_out10MHz = ~clk_out10MHz;    // 10 MHz
    always #62.5  clk_out8MHz  = ~clk_out8MHz;     // 8 MHz
endmodule

```

Testbench Simulation (Tanner)

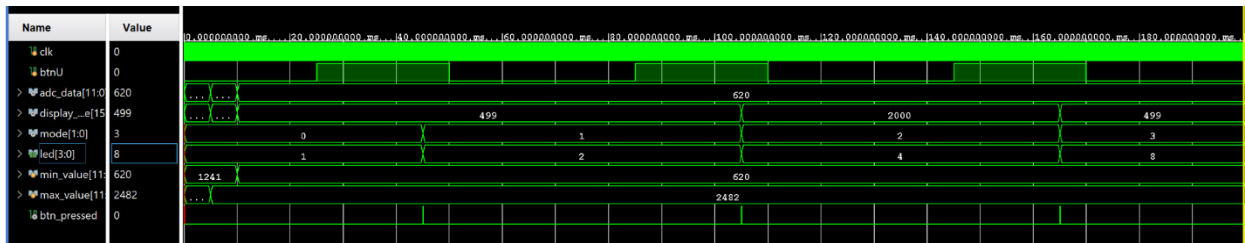


Figure 4: Simulation of Voltmeter

Working Design

Below are images that show the functionality of the board at different voltages that are being provided to the Voltmeter. The device was showing an accuracy that we were happy with. We found it to be within $\pm 10 \mu V$ of what our DC supply was giving as an output.

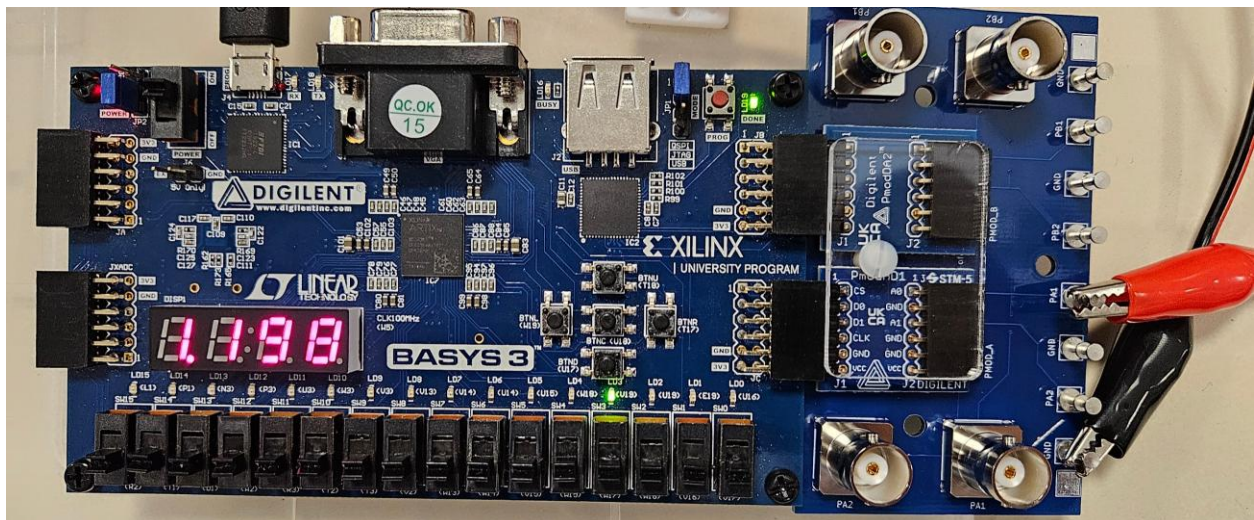


Figure 5: Voltmeter Reading 1.2V

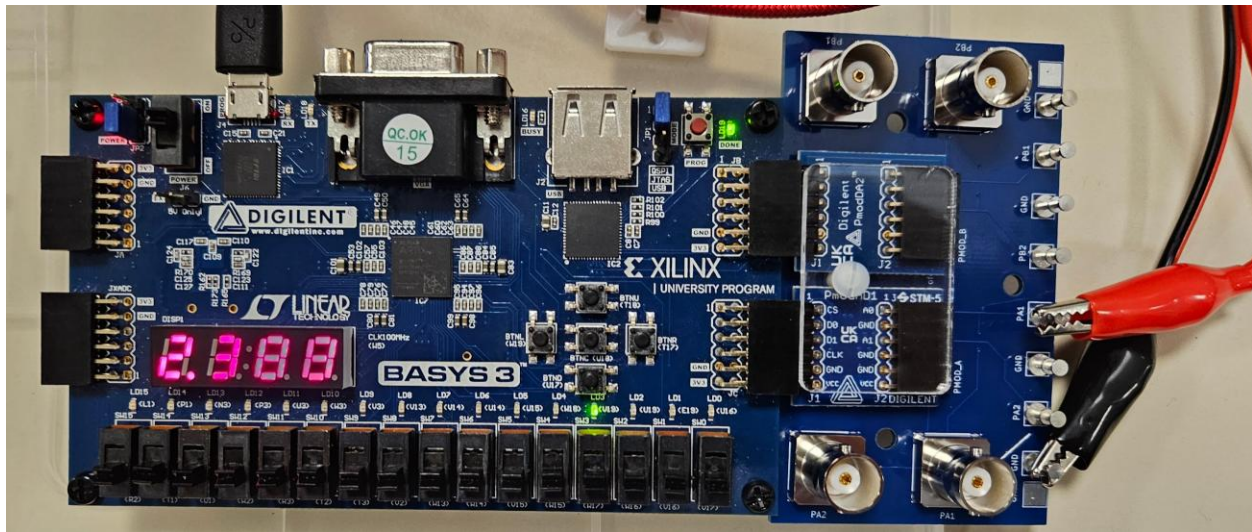


Figure 6: Voltmeter Reading 2.4V

Conclusion

Through this project, our team successfully designed and implemented an FPGA-based digital voltmeter using the Basys3 development board and a 12-bit ADC. Throughout the design process, we gained experience developing individual hardware modules, including a clock generator, ADC interface, finite state machine, memory controller, seven-segment display driver, and top-level integration module. We also learned the importance of modular design, simulation, and hardware verification when developing complex digital systems. One of the greatest challenges was figuring out all of the different modules that we needed to make the system run. Splitting parts up made it easy on the top module to put them all together into one simple system. By creating a testbench and comparing the hardware output to a known input voltage from a DC power supply, we were able to identify any issues.

Overall, this project fulfilled the purposes of the lab by requiring us to define and implement a complex digital system, collaborate effectively as a team, integrate multiple digital design concepts into a single project, and communicate our design through testing, documentation, and demonstration. The experience provided valuable practice in FPGA design, hardware debugging, and system integration that will be beneficial in future engineering projects.